fire_analysis_notebook

September 4, 2025

1 Fire analysis script

The goal of this script is to complete a regression where region of interest type, saliency, and other features are used to predict fixation behavior.

1.1 1. Load eye movement data

This cell loads the monkey's raw eye movement data from a CSV file.

[1]:
```
import pandas as pd

eye_df = pd.read_csv("RawData_AllAnimals.csv")
eye_df = eye_df[eye_df['ROI'] != "OffScreen"] #remove rows where ROI is␣
 ↪"OffScreen"
eye_df = eye_df[eye_df['ImageType'] == "F"] #Only keep rows where ImageType is␣
 ↪"F"
#eye_df = eye_df[eye_df['FixDur'] >= 100] #Only fixations over X ms
```

1.2 2. Load ROI masks and saliency maps

This cell loads all ROI type/index masks and saliency maps into dictionaries for fast lookup by image name.

[2]:
```
import numpy as np
import os

# Load ROI masks
roi_type_dict = {}
roi_index_dict = {}

csv_names = os.listdir("csv_output")
csv_names.remove("mean_saliency.csv")  # Exclude mean_saliency if present

for fname in os.listdir("contour_masks"):
    if "roi_type" in fname:
        key = fname.split("_roi_type")[0]
        roi_type_dict[key] = np.load(os.path.join("contour_masks", fname))
    elif "roi_index" in fname:
```

```
        key = fname.split("_roi_index")[0]
        roi_index_dict[key] = np.load(os.path.join("contour_masks", fname))

# Calculate the size (number of pixels) of each ROI in each image

roi_sizes = {}

for img_name, roi_index_mask in roi_index_dict.items():
    unique, counts = np.unique(roi_index_mask, return_counts=True)
    roi_sizes[img_name] = dict(zip(unique, counts))

# Example: print ROI sizes for the first image
first_img = list(roi_sizes.keys())[0]
print(f"ROI sizes for {first_img}: {roi_sizes[first_img]}")

# Load saliency maps from CSV files into a dictionary with image base names as␣
 ↪keys
saliency_dict = {}
for fname in csv_names:
    if fname.endswith("_saliency.csv"):
        key = fname.split("_saliency")[0]
        # Try reading as CSV, skip header if present
        try:
            arr = np.loadtxt(os.path.join("csv_output", fname), delimiter=',')
        except Exception:
            arr = pd.read_csv(os.path.join("csv_output", fname), header=None).
 ↪values
        saliency_dict[key] = arr
```

```
ROI sizes for FStil01.jpg: {0: 1694055, 2: 83428, 6: 204523, 7: 30584, 8: 41445,
9: 19565}
```

1.3 3. Join eye movement data with image features

This cell creates a new DataFrame where each fixation is enriched with ROI type, ROI index, and saliency at the fixation location.

[3]:
```
def get_base_name_images(stimuli):
    # Always use .jpg for lookup, regardless of original extension
    base = os.path.splitext(os.path.basename(stimuli))[0]
    return base + ".jpg"

features = []

#go through each row in eye_df and extract features
for idx, row in eye_df.iterrows():
    img_key = get_base_name_images(row['Stimuli'])
    x, y = int(row['XPos']), int(row['YPos'])
```

```
    # Skip if image not found or fixation out of bounds
    if img_key not in roi_type_dict or img_key not in saliency_dict:
        continue

    roi_type = roi_type_dict[img_key]
    roi_index = roi_index_dict[img_key]
    saliency = saliency_dict[img_key]

    if y >= roi_type.shape[0] or x >= roi_type.shape[1]:
        continue

    features.append({
        "Subject": row["Subject"],
        "Stimuli": row["Stimuli"],
        "FixStart": row["FixStart"],
        "FixEnd": row["FixEnd"],
        "FixDur": row["FixDur"],
        "XPos": x,
        "YPos": y,
        "ROI": row["ROI"],
        "Block": row["Block"],
        "Trial": row["Trial"],
        "ImageType": row["ImageType"],
        "Species": row["Species"],
        "SubjectName": row["SubjectName"],
        "roi_type": roi_type[y, x],
        "roi_index": roi_index[y, x],
        "saliency": saliency[y, x],
        "area": roi_sizes[img_key].get(roi_index[y, x], 0)  # Get area size or␣
↪0 if not found
    })

fixation_features_df = pd.DataFrame(features)
fixation_features_df.head()
```

[3]:

	Subject	Stimuli	FixStart	FixEnd	FixDur	XPos	YPos	ROI	Block	Trial	ImageType	Species
0	Cheyenne_20240702_1420	FStil09.png	232.713	392.695	163.256	1527	561	NonFire	1	Cheyenne_Block1_20240702_1420.xlsx	F	Baboon
1	Cheyenne_20240702_1420	FStil09.png	409.259	645.983	239.965	1093	378	Fire	1	Cheyenne_Block1_20240702_1420.xlsx	F	Baboon
2	Cheyenne_20240702_1420	FStil09.png	655.942	869.192	216.689	1193	525	NonFire	1	Cheyenne_Block1_20240702_1420.xlsx	F	Baboon
3	Cheyenne_20240702_1420	FStil09.png	899.287	1055.843	159.995	1664						
4	Cheyenne_20240702_1420	FStil04.png	83.609	213.495	133.334	929						

```
3   123  NonFire      1  Cheyenne_Block1_20240702_1420.xlsx         F  Baboon
4   579  NonFire      1  Cheyenne_Block1_20240702_1420.xlsx         F  Baboon

  SubjectName  roi_type  roi_index    saliency     area
0    Cheyenne         0          0   52.848877  1563409
1    Cheyenne         1          3  171.167404    96632
2    Cheyenne         0          0  148.384018  1563409
3    Cheyenne         0          0   36.488598  1563409
4    Cheyenne         0          0   40.138653  1639293
```

[4]:
```
# Calculate matches and mismatches between ROI label and roi_type
matches = fixation_features_df[(fixation_features_df["ROI"] == "Fire") &␣
 ↪(fixation_features_df["roi_type"] == 1)]
mismatches = fixation_features_df[(fixation_features_df["ROI"] == "Fire") &␣
 ↪(fixation_features_df["roi_type"] != 1)]

total_fire = len(fixation_features_df[fixation_features_df["ROI"] == "Fire"])
percent_match = len(matches) / total_fire * 100 if total_fire > 0 else 0
percent_mismatch = len(mismatches) / total_fire * 100 if total_fire > 0 else 0

print(f"Percent where ROI == 'Fire' and roi_type == 1: {percent_match:.2f}%")
print(f"Percent where ROI == 'Fire' and roi_type != 1: {percent_mismatch:.2f}%")

# Drop rows with missing values in key columns
fixation_features_df = fixation_features_df.dropna(subset=["FixDur", "XPos",␣
 ↪"YPos", "ROI", "roi_type"])

# Remove outliers in fixation duration define with Ben
# fixation_features_df = fixation_features_df[(fixation_features_df["FixDur"]␣
 ↪>= 50) & (fixation_features_df["FixDur"] <= 1000)]

# Convert columns to correct types
# fixation_features_df["Trial"] = fixation_features_df["Trial"].astype(int) not␣
 ↪needed because not used in analysis
# fixation_features_df["Block"] = fixation_features_df["Block"].astype(int)
fixation_features_df["SubjectName"] = fixation_features_df["SubjectName"].
 ↪astype(str)

# Remove outliers in fixation duration
mean_fix_dur = fixation_features_df["FixDur"].mean()
std_fix_dur = fixation_features_df["FixDur"].std()
fixation_features_df = fixation_features_df[(fixation_features_df["FixDur"] >=␣
 ↪50) & (fixation_features_df["FixDur"] <= mean_fix_dur + std_fix_dur * 3)]
```

```
Percent where ROI == 'Fire' and roi_type == 1: 64.25%
Percent where ROI == 'Fire' and roi_type != 1: 35.75%
```

1.4 4. Logistic Regression: Predicting fire region fixations

This cell shows how to use the joined DataFrame to predict whether a fixation falls on a fire region (roi_type == 1) using saliency and fixation duration as predictors.

[14]:
```
from sklearn.linear_model import LogisticRegression

# Create binary target: 1 if fire region, 0 otherwise
fixation_features_df["is_fire"] = (fixation_features_df["roi_type"] == 1).
 ↪astype(int)
X = fixation_features_df[["saliency", "FixDur", "area"]]
y = fixation_features_df["is_fire"]

model = LogisticRegression()
model.fit(X, y)

print("Regression coefficients:", model.coef_)
print("Odds ratios:", np.exp(model.coef_))
```

```
Regression coefficients: [[ 2.64528938e-02 -2.65958099e-03 -4.95069031e-06]]
Odds ratios: [[1.02680588 0.99734395 0.99999505]]
```

1.5 5. Machine Learning Logistic Regression

In this cell, we apply a machine learning approach using logistic regression to predict whether a fixation falls on a fire region (roi_type == 1) based on saliency and fixation duration. The data is split into training and test sets to evaluate the model's predictive accuracy on unseen data.

[28]:
```
from sklearn.model_selection import LeaveOneGroupOut, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# Features and target
X = fixation_features_df[["saliency", "FixDur", "area"]]
y = fixation_features_df["is_fire"]
groups = fixation_features_df["SubjectName"]

# Pipeline with scaling + logistic regression
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=10000, random_state=0))
])

# Leave-One-Subject-Out CV
logo = LeaveOneGroupOut()
scores = cross_val_score(pipe, X, y, cv=logo, groups=groups, scoring="accuracy")

print("Mean accuracy across subjects:", scores.mean())
```

```
Mean accuracy across subjects: 0.9418473972357879
```

1.6 6. OLS Regression Predicting Fixation Duration

In this cell, we predict fixation duration from saliency, area, and if its fire or not

[29]:
```python
import statsmodels.api as sm

# Predict FixDur using saliency, area, and whether the fixation is on fire
X = fixation_features_df[["area", "is_fire"]]
X = sm.add_constant(X)  # Adds intercept
y = fixation_features_df["FixDur"]

model = sm.OLS(y, X)
result = model.fit()
print(result.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                 FixDur   R-squared:                       0.001
Model:                           OLS   Adj. R-squared:                  0.000
Method:                Least Squares   F-statistic:                     3.668
Date:               Thu, 04 Sep 2025   Prob (F-statistic):             0.0256
Time:                       15:19:04   Log-Likelihood:                -76687.
No. Observations:              12243   AIC:                         1.534e+05
Df Residuals:                  12240   BIC:                         1.534e+05
Df Model:                          2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        235.9994      4.036     58.472      0.000     228.088     243.911
area        2.862e-06   2.59e-06      1.106      0.269   -2.21e-06    7.94e-06
is_fire       10.3708      4.335      2.392      0.017       1.873      18.868
==============================================================================
Omnibus:                     3629.269   Durbin-Watson:                   1.727
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            10223.400
Skew:                           1.574   Prob(JB):                         0.00
Kurtosis:                       6.183   Cond. No.                     6.72e+06
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The condition number is large, 6.72e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
```

1.7 7. Linear mixed-effects model (LME)

In this cell, we predict fixation duration from saliency, area, and if its fire or not while accounting for within-subject variability.

[31]:
```
import statsmodels.formula.api as smf

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
#this is to z-score the saliency and area we get issues because they are on␣
 ↪different scales
fixation_features_df[["saliency_scaled", "area_scaled"]] = scaler.
 ↪fit_transform(fixation_features_df[["saliency", "area"]])
# Mixed-effects linear regression: FixDur ~ saliency + area + is_fire +␣
 ↪(1|Subject)
model = smf.mixedlm(
    "FixDur ~ saliency_scaled + area_scaled + is_fire",
    fixation_features_df,
    groups=fixation_features_df["SubjectName"]
)
result = model.fit()
print(result.summary())
```

```
            Mixed Linear Model Regression Results
============================================================
Model:               MixedLM  Dependent Variable:  FixDur
No. Observations:    12243    Method:              REML
No. Groups:          11       Scale:               14281.6920
Min. group size:     695      Log-Likelihood:      -75952.3337
Max. group size:     1820     Converged:           Yes
Mean group size:     1113.0
------------------------------------------------------------
                 Coef.   Std.Err.    z    P>|z|  [0.025  0.975]
------------------------------------------------------------
Intercept        252.520   13.670 18.473 0.000 225.728 279.312
saliency_scaled    9.091    1.390  6.542 0.000   6.368  11.815
area_scaled        2.836    1.760  1.611 0.107  -0.614   6.286
is_fire           -1.257    4.518 -0.278 0.781 -10.112   7.599
Group Var       2027.522    7.616
============================================================
```

1.8 7 (cont.). Linear mixed-effects model (LME) Step-wise comparison

In this cell, we predict fixation duration from saliency, area, and if its fire or not while accounting for within-subject variability. To see if is_fire improves the model we use step-wise Linear mixed-effects models.

[34]:
```
# Import packages
import pandas as pd
import statsmodels.formula.api as smf
from sklearn.preprocessing import StandardScaler
from scipy.stats import chi2

# Assume fixation_features_df is already loaded with columns:
# 'FixDur', 'saliency', 'area', 'is_fire', 'SubjectName'

# 1 Z-score the continuous predictors
scaler = StandardScaler()
fixation_features_df[["saliency_scaled", "area_scaled"]] = scaler.fit_transform(
    fixation_features_df[["saliency", "area"]]
)

# 2 Fit base model (without is_fire)
base_model = smf.mixedlm(
    "FixDur ~ saliency_scaled + area_scaled",
    fixation_features_df,
    groups=fixation_features_df["SubjectName"]
)
base_result = base_model.fit(reml=False)  # ML is required for likelihood ratio␣
 ↪test

# 3 Fit full model (with is_fire)
full_model = smf.mixedlm(
    "FixDur ~ saliency_scaled + area_scaled + is_fire",
    fixation_features_df,
    groups=fixation_features_df["SubjectName"]
)
full_result = full_model.fit(reml=False)

# 4 Print summaries
print("=== Base Model Summary ===")
print(base_result.summary())
print("\n=== Full Model Summary ===")
print(full_result.summary())

# 5 Likelihood ratio test for incremental effect of is_fire
lr_stat = 2 * (full_result.llf - base_result.llf)
df_diff = full_result.df_modelwc - base_result.df_modelwc
p_value = chi2.sf(lr_stat, df_diff)

print(f"\nLikelihood Ratio Test for 'is_fire':")
print(f"LR statistic = {lr_stat:.3f}, df = {df_diff}, p-value = {p_value:.3f}")
```

```
=== Base Model Summary ===
            Mixed Linear Model Regression Results
```

```
============================================================
Model:               MixedLM  Dependent Variable:  FixDur
No. Observations:    12243    Method:              ML
No. Groups:          11       Scale:               14278.2488
Min. group size:     695      Log-Likelihood:      -75960.4554
Max. group size:     1820     Converged:           Yes
Mean group size:     1113.0
------------------------------------------------------------
                 Coef.   Std.Err.   z    P>|z|  [0.025  0.975]
------------------------------------------------------------
Intercept        252.201   13.007 19.389 0.000 226.708 277.695
saliency_scaled    8.926    1.258  7.097 0.000   6.461  11.391
area_scaled        3.179    1.253  2.538 0.011   0.724   5.634
Group Var       1846.921    6.639
============================================================


=== Full Model Summary ===
            Mixed Linear Model Regression Results
============================================================
Model:               MixedLM  Dependent Variable:  FixDur
No. Observations:    12243    Method:              ML
No. Groups:          11       Scale:               14278.1565
Min. group size:     695      Log-Likelihood:      -75960.4168
Max. group size:     1820     Converged:           Yes
Mean group size:     1113.0
------------------------------------------------------------
                 Coef.   Std.Err.   z    P>|z|  [0.025  0.975]
------------------------------------------------------------
Intercept        252.513   13.056 19.340 0.000 226.923 278.103
saliency_scaled    9.090    1.390  6.542 0.000   6.367  11.814
area_scaled        2.836    1.760  1.611 0.107  -0.614   6.285
is_fire           -1.255    4.518 -0.278 0.781 -10.110   7.599
Group Var       1847.233    6.640
============================================================


Likelihood Ratio Test for 'is_fire':
LR statistic = 0.077, df = 1, p-value = 0.781
```